CMPT 276

Phase 3 Report

M. Vivan Prasad, Yoonsang You, Anthony Yuen, Artyom Borodovitsyn

---

Introduction

The visual update and code refactoring in phase 3 significantly improved the data structures and addressed several issues with enemies, spawning and animations. We revised key data structures once again to eliminate unnecessary dependencies and optimize performance. A key design approach used was the Entity Component System with Tiles (ECS), integrated with tiles. The ECS framework, with components like rect, hitbox, and AnimationPlayer, allowed for more flexible and maintainable code, eliminating hard-coded bugs previously present in the implementation of enemies and entity generation. By organizing entities into components, we were able to create an easily extendable game architecture for further development and improving overall game performance and player experience.

Unit Tested Features

For the purpose of this project, the focus of testing was based on class functionality rather than the user interface graphics. The following features were identified for unit testing:

- **AnimationPlayer**: This class holds a list of Animation objects, which define the animation frames and their durations (potentially looping). Unit tests will ensure that animations are loaded and executed correctly based on the game logic.
- **Panels**: The different UI panels in the game, such as the main menu, pause menu, and game over screens, require testing to ensure they display and function correctly based on user input and game state.
- **Object:** Rect and Vector classes assist in managing tile positioning and geometry. Unit testing will be done for vector operations and collision detection.
- **Generator**: The EnemyGenerator and RewardGenerator classes need testing to verify that they correctly instantiate enemies and rewards.
- **Music/SFX Classes**: These classes are responsible for playing background music (BGM) and sound effects (SFX). Unit tests will check that sound events trigger correctly during gameplay, ensuring proper audio feedback.

Important Interactions Between Different Components

For our game, several key interactions occur between different components. These interactions are important for the functionality and integration of various game features. Below are identified interactions and the associated test cases:

1. **ResourceLoader & associated tile generator classes or audio classes:** The ResourceLoader is responsible for loading essential game resources, including sprite sheets, audio files, animations, and level data. The Tile and Generator classes depend on these resources to display visuals and manage gameplay elements. For example, the Tile class uses sprites loaded by the ResourceLoader to draw tiles in the game world, and the Generator class uses the resources to spawn enemies and rewards.
   Test Cases:
   - TestResourceLoader(): verifies that all necessary resources are loaded correctly and available to other classes, ensuring proper integration between ResourceLoader and Tile/Generator.
   - TestAudio.testMusicPlay(): Verifies that music files are loaded and played correctly.
   - TestAudio.testSFXPlay(): Ensures that sound effects are properly loaded and played
   - TestAudio.testSFXLoop(): Checks that sound effects can loop correctly.
   - TestAudio.testSFXStop(): Verifies the ability to stop sound effects.
2. **EnemyGenerator and GamePanel:** The EnemyGenerator class is responsible for spawning enemies in the game. It interacts with the GamePanel to place the enemies on the screen and update them during the game loop. The EnemyGenerator ensures that the correct number of enemies (e.g. guards, dogs, cameras, lasers) are created and managed.
   Test Cases:
   - TestEnemyGenerator.testSpawnGuards() or Dogs or Cameras or Lasers: Verifies that the correct number of guards/dogs/cameras is spawned.
   - TestEnemyGenerator.testUpdateEnemies(): Updates the state of all enemies and checks for collisions with the player.
   - TestEnemyGenerator.testAddSpeed(): Verifies that enemy speed can be increased correctly.
3. **RewardGenerator and GamePanel:** The RewardGenerator class handles the spawning and management of rewards in the game. It interacts with the GamePanel to place these rewards in the game, check for interactions, and remove expired rewards. This ensures that players can collect rewards and that expired ones are removed from the game state.
   Test Cases:
   - TestRewardGenerator.testSpawnRewards(): Verifies that rewards are spawned correctly.
   - TestRewardGenerator.testRemoveExpiredRewardsPickup(): Ensures expired rewards are removed after pickup.
   - TestRewardGenerator.testUpdateRewards(): Verifies that rewards are updated properly during the game.
4. **TileManager and GamePanel:** The TileManager is responsible for managing tiles in the game, such as the floors, walls, and other elements. It interacts with the GamePanel to update the game with the correct tiles and determine the correct draw order for objects.
   Test Cases:
   - TestTileManager.testGetEmptyTile(): Verifies that the TileManager can provide empty tiles

- TestTileManager.testNextEmptyTile(): Ensures the TileManager can correctly get the next empty tile.
- TestTileManager.testGetFloorTiles(): Verifies that floor tiles are correctly retrieved by the TileManager.

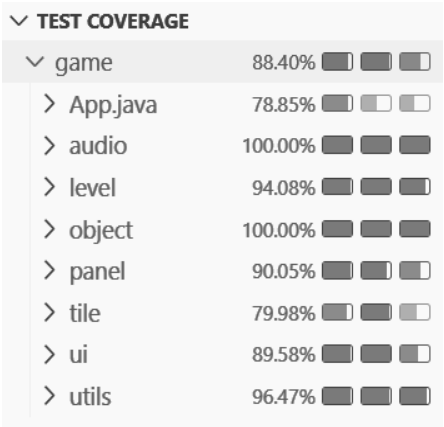
5. **Rect and Vector Classes for Positioning and Movement:** The Rect and Vector classes are used to manage the position, sizes, and movement of objects. These classes interact with entities to handle their movement and collision detection.
   Test Cases:
   - TestRect.testConstructorWithCoordinates(): Verifies the correct initialization of Rect objects with specific coordinates.
   - TestVector.testAdd(): Ensures that vectors can be correctly added together for movement calculations.
   - TestRect.testIsTouching(): Verifies that Rect objects can correctly detect collisions with each other.

Strategies for Quality of Test Cases

- Feature Identification and Isolation: We first identified which features of the game required testing, and we isolated those features from other parts of the code. This isolation allowed our tests to focus directly on specific functionalities, making them more precise and easier to debug.
- Modular Approach: We used a modular testing approach, where each test case was tightly coupled with the class it was testing. This structure made it easier to maintain and extend tests in the future. For instance, TestAnimationPlayer only tests the AnimationPlayer class, ensuring that the tests remain simple and focused.
- Edge Case Simulation: We specifically designed test cases to simulate edge cases—scenarios that push the limits of the system. For example, we tested how the game reacts when there are very few resources left, or when animations are triggered in quick succession. This ensures that the game functions well even under extreme conditions and handles errors gracefully.

Code Coverage Report

The total observed coverage from the tests was 88.40% coverage overall. This indicates that the majority of the code in the project is covered by unit tests, but there are methods not intentionally tested for reasons mentioned earlier. Breaking the coverage down: 1,536/1,660 (92.53%) statements were covered, 308/326 (94.48%) methods were covered, and 473/635 (74.49%) of branches were covered. This coverage reflects a solid level of coverage, though not all potential branches are tested.



The lower branch coverage can be attributed to the UI, Tile and App classes, specially the branching in the draw() methods for hitbox rendering, screen image displays and draw order in TileManager class. Since we are focusing on functional testing and not UI graphics testing, these branches were omitted from being covered as they do not impact the core functionality of the game. This includes the draw() method across various game components.

While the current coverage is considerably high, focusing on the remaining gaps will help ensure the robustness and stability of the game as we continue to further develop the game.

Insights Gained from Writing and Running Tests

Writing and running tests provided valuable insights into the functionality and code quality of each class of the game. Through the tests, we identified bugs and unexpected results that were not apparent during development, such as minor inconsistencies in the timing class discussed in *Bugs and Issues Found*. Our tests often also lead to improved software design. When testing our components, we have to think about how components will interact and how to make them testable. This encourages modular and loosely coupled design, which is easier to maintain and extend. By breaking down the game into smaller and testable components, we were able to identify the issue without tracing the entire application. This made refactoring our classes with ease, ensuring that future changes do not introduce new issues into the game.

Changes to Production Code

During the testing phase, additional changes were made to the production code. These changes primarily focused on refining the game mechanics, improving visual effects, and optimizing the internal structure of the game. The following updates were introduced:

1. **Intro Fade Animation** - A smooth fade-in animation was added to the introduction of the game, improving the visual appeal and providing a more polished user experience.
2. **Smooth Camera Animations** - In order to improve the visual experience during gameplay, smooth camera transitions were implemented. The camera now smoothly follows the player's movements instead of jumping abruptly between positions.
3. **New Reward Pickup and Floating Animations** - A set of new animations was introduced for the collection of rewards. The rewards now have a floating effect that makes them visually more appealing when picked up.
4. **Rewards Spawn Expiring Indicator** - A new indicator was added to show when a reward is about to expire or despawn. The reward will flash faster over their spawn time duration. The faster the flash signals that the reward is about to expire.
5. **Draw Ordering with Layer Enum** - The draw order of objects in the game was refined by implementing a new Layer enum. This enum categorizes different in-game objects (such as walls, enemies, and rewards) into specific layers, ensuring that the objects are drawn in the correct order. This resolved issues where entities, such as enemies, would appear behind walls or other obstacles, preventing visual glitches and improving the overall rendering process.

6. **New Rect and Vector Classes** - Two new utility classes, Rect and Vector, were introduced to simplify the management of tile positioning and geometry in the game. These classes contain useful methods for vector operations and collision detection, allowing for cleaner and more efficient calculations related to movement, boundaries, and interactions between game objects. These changes also improved the readability of the code and made future modifications easier to implement.
7. **Tile Class as the Base Class for In-Game Tiles** - The Tile class was created as a base class for all in-game tiles. This refactor allowed various tile objects, such as walls, rewards, and entities, to inherit from the Tile class, promoting better code reuse and organization. Previously, the abstract Sprite class was underused and did not contribute effectively to the tile system. With the new Tile class structure, we can now more easily manage different tile types and their behavior in the game.
8. **New Label and Container Classes for In-Game UI Handling** - To simplify the handling of in-game user interface (UI) elements, new Label and Container classes were introduced. These classes are responsible for holding and arranging UI components like text labels and containers for buttons or other interactive elements within the game. This reduced the redundancy in our code. For example, our code was previously hardcoded for each panel. The introduction of these classes allowed for a more modular and flexible approach to UI management, improving the code's scalability as new features are added.
9. **Character Class for Entity Movement and Basic Movement Animations** - A class that extends the Entity class. Character class represents a playable character in the game. Characters can have directions and speed. This maintains the class hierarchy between enemies and the player, and potentially sets a basis for further modifications to the playable character (e.g. obtaining new rewards to enhance the playable character as the levels get more difficult).
10. **Generator class component of EnemyGenerator and RewardGenerator** - To simplify the EnemyGenerator and RewardGenerator, we added a Generator class which is solely responsible for spawning and managing the tiles of the instantiated GamePanel. This further allows future updates to be implemented such as new enemies for new levels.
11. **TileManager** - Responsible for generating and managing the tiles in the game, including floors, walls, and available tiles for spawning. It loads map data from resources and separates tiles based on their types and properties. This class also handles drawing the tiles in the correct order to ensure proper layering in the game. This class reduces the complexity of instantiating the tile and improves the code clarity and class functionality.

Bugs and Issues Found

Several bugs were identified and resolved, leading to significant improvements in the overall quality of the code. Through careful testing, we were able to uncover issues that affected gameplay, performance, and visual elements, allowing us to make necessary fixes and optimizations. These changes not only enhanced the user experience but also improved the maintainability and structure of the code, ensuring better stability and scalability for future development.

1. **Mac Compatibility -** The game had issues displaying button text on macOS. The game displays the menu with blank white buttons in both the main menu and the end of the game. This may be due to font rendering issues particular for Mac users.
2. **Draw Order and Layering Issues** - The incorrect layering of game objects, particularly enemies and walls, led to visual glitches where enemies were sometimes drawn behind or inside walls.
3. **Timer Class variability -** The class is working as intended, however, the timer may return slightly inaccurate results (e.g. the result would be 0 to 3ms off from the expected output). This could lead to inaccurate scores by an insignificant amount. When the player is caught on level 1, the last score should be 0, but rarely it would show a value of 1. This issue was addressed by refactoring getTimeLeft() with getSecondsLeft(), which resulted in less variability and more accurate results by improving the synchronization between the timer's start and the game's frame updates. The issue remains as an innate problem due to the timer running asynchronously.
4. **End Game/Next Level Panel Not Displaying Correctly** - There is a random occurrence where the end game or next level panel does not appear as expected. During these instances, the game seems to freeze, and the expected panel is not visible. Upon investigation, it was found that the panel does exist but rendered behind the GamePanel.

Conclusion

The testing phase helped identify and fix several bugs and improve the overall quality of the game. Through a combination of unit and integration tests, we were able to ensure the functionality of individual features and smooth integration of each component. The changes made during this phase improved our game's maintainability by refactoring large classes and reducing coupling between classes. From this phase, we addressed multiple bugs found, improved code quality, and significantly strengthened the game and its foundation for further development.